Computer Vision Assignment: Pet Classification

Name: Ioannis Petrakis
ID: 1115201900155

5/2/2026

1 Introduction: Design and Architectural Choices

This report presents a comparative study between a Deep Learning approach (Convolutional Neural Networks) and a Traditional Computer Vision pipeline (Bag-of-Visual-Words) for the fine-grained classification of six pet breeds from the Oxford-IIIT Pet Dataset. A critical constraint of this study is the limited data availability ($\sim$160 images per class) and the requirement for low computational cost.

1.1 Deep Learning Pipeline: The Dual-Head U-Net

To address the challenge of data scarcity, we designed a custom **Multi-Task Learning (MTL) architecture**. Instead of a standard classifier (e.g., ResNet), we implemented a modified **U-Net** with a shared encoder and two parallel heads:

1. **Segmentation Head:** A decoder path that reconstructs the spatial binary mask (Foreground vs. Background).
2. **Classification Head:** A parallel branch attached to the bottleneck that applies *Adaptive Average Pooling* followed by a fully connected layer for the 6-class prediction.

Design Rationale: The primary motivation for this dual-head design is **regularization via auxiliary tasks**. By forcing the encoder to minimize the segmentation loss (L_{seg}), the network is compelled to learn robust spatial features (shapes, boundaries, textures) that correspond to the animal itself, rather than overfitting to background noise.

- **Shallow Architecture:** To prevent overfitting on the small dataset, the network depth was restricted to 2 encoder blocks.
- **Unified Loss Function:** The model is optimized using a weighted cumulative loss:

$$L_{total} = \lambda \cdot L_{seg} + L_{cls} \tag{1}$$

  Here, λ acts as a hyperparameter to balance the gradient contribution from the segmentation task.
- **Activation Functions:** We selected **ReLU** (Rectified Linear Unit) as the non-linear activation function. Unlike Sigmoid or Tanh, ReLU does not saturate for positive values, effectively mitigating the *vanishing gradient problem* and accelerating convergence in deep networks .

Regularization & Optimization Protocol: Given the small training set, preventing overfitting was paramount. We applied **Dropout** ($p = 0.2$) after each convolutional block and **Weight Decay** ($1e^{-4}$) in the Adam optimizer. Training was capped at 30 epochs to limit computational cost. To select the optimal model, we employed a **Best Model Saving** strategy, monitoring the validation accuracy at every epoch rather than relying solely on the final epoch's weights.

1.2 Traditional Pipeline: SIFT and Bag-of-Visual-Words

As a baseline, we implemented a feature engineering pipeline that does not require end-to-end training of visual filters.

1. **Feature Extraction (SIFT):** We utilized the Scale-Invariant Feature Transform (SIFT) to extract local keypoint descriptors. SIFT was selected for its robustness to scale and rotation changes, which is crucial given the unconstrained poses of the pets.

2. **Visual Vocabulary (K-Means):** To aggregate the variable number of descriptors per image into a fixed-length vector, we constructed a visual codebook (Bag-of-Words) using K-Means clustering on a subset of $M = 40,000$ descriptors.

3. **Histogram Encoding:** Each image was represented as a frequency histogram of visual words. We applied **L1 Normalization** to treat these histograms as probability distributions, ensuring invariance to the total number of keypoints detected.

4. **Classifier (MLP):** A shallow Multi-Layer Perceptron (2 hidden layers) was trained on these histograms.

The comparison between these two methods aims to investigate whether "hand-crafted" local features (SIFT) can outperform a deep CNN when training data is insufficient for learning generalizable filters from scratch.

2 Implementation Details

Our implementation was developed using **PyTorch** for deep learning and **OpenCV** for traditional image processing. The pipeline consists of distinct modules corresponding to the assignment tasks.

2.1 Data Preparation and Preprocessing (Block 1)

The dataset setup involved filtering the *Oxford-IIIT Pet Dataset* to retain only the six target breeds. All images and corresponding segmentation masks were resized to fixed dimensions of 256×256 pixels.

- **Mask Remapping:** The original trimaps (1:Foreground, 2:Background, 3:Border) were remapped to standard semantic segmentation labels: 0 for Background, 1 for Foreground, and 255 for the Border. The label 255 is explicitly handled as an `ignore_index` in the loss function to prevent ambiguous boundary pixels from affecting the gradient.

- **Splitting:** We employed a fixed seed (42) to generate a reproducible 80/20 split for training and validation, ensuring fair comparison across all experiments.

2.2 Neural Pipeline: Implementation and Training (Blocks 3-4)

Architecture: We implemented a `DoubleHeadUNet` (Block 3). The encoder utilizes standard Convolution-BatchNorm-ReLU blocks with MaxPooling. To adhere to the low-complexity constraint, the network depth was limited to two down-sampling stages.

- The **Segmentation Head** uses bilinear upsampling and skip connections to recover spatial resolution.
- The **Classification Head** applies `AdaptiveAvgPool2d(1)` to the bottleneck features, flattening the spatial dimensions before passing the vector through a linear classifier.

Training Protocol (Block 4): The model was trained using the **Adam** optimizer ($lr = 0.001$) for a maximum of 30 epochs. We implemented a custom training loop to minimize the joint loss:

$$\mathcal{L} = \lambda \cdot \mathrm{CE}_{seg} + \mathrm{CE}_{cls} \tag{2}$$

We performed a grid search for the weighting parameter $\lambda \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0\}$. The optimal model was selected based on the highest Validation Accuracy (Best Model Saving strategy) to mitigate overfitting.

2.3 Traditional Pipeline: Bag-of-Visual-Words (Blocks 2 & 5)

We implemented a manual Feature Engineering pipeline without using high-level wrapper classes, explicitly controlling the clustering process.

Feature Extraction: We employed the SIFT algorithm (`contrastThreshold` $= 0.03$) to detect scale-invariant keypoints. **Vocabulary Construction:** We aggregated descriptors from the entire training set and randomly sampled $M = 40,000$ vectors to maintain computational feasibility. **K-Means clustering** was applied to generate a visual codebook of size K. **Encoding:** For every image, extracted descriptors were assigned to the nearest visual word using L2 distance. The resulting frequency histogram was **L1-normalized** to function as a probability distribution. **Hyperparameter Search:** We executed the full pipeline for three codebook sizes $K \in \{300, 400, 500\}$, selecting the K with the highest validation accuracy.

2.4 Robustness Verification (Block 6)

To verify that the models learned to recognize the animal rather than the environment, we constructed a **Noisy Background Dataset**. **Methodology:** Using bitwise operations guided by the ground-truth masks, we isolated the foreground (pet) and replaced the background pixels with Gaussian noise ($\mu = 127, \sigma = 40$) smoothed by a blur filter. Both the optimized CNN and the best MLP were evaluated on this synthetic set without retraining.

2.5 Mask-Guided Vocabulary (Block 7 - Bonus)

We extended the traditional pipeline by utilizing semantic segmentation masks during the codebook construction phase. **Protocol:** Instead of using all descriptors, the K-Means codebook was built using *only* SIFT descriptors that spatially fell within the foreground mask (Pet + Boundary). We tested two variants:

1. **Experiment 1 (Hybrid):** Building the vocabulary from Foreground-Only features, but encoding the full image during training and testing.
2. **Experiment 2 (Foreground-Focus):** Building the vocabulary from Foreground-Only features and encoding only foreground descriptors during training, to simulate an "ideal" attention mechanism.

2.6 Statistical Evaluation and Visualization (Block 8)

To finalize the assessment, we implemented a comprehensive evaluation module:

- **Quantitative Metrics:** We computed the final **Correct Classification Rate (CCR)** and generated **Confusion Matrices** for both the CNN and MLP on the Test Set. For segmentation, we calculated the **Global mIoU** by accumulating intersections and unions across the entire dataset.
- **Qualitative Assessment:** We generated side-by-side visualizations of input images, ground-truth masks, and predicted masks to visually inspect the CNN's ability to localize the pet boundaries.
- **Learning Curves:** Training history (Loss, Accuracy, mIoU) was plotted to diagnose convergence behavior and potential overfitting.

3 Experimental Results

In this section, we present the quantitative performance of the proposed methods on the test set, analyze the impact of hyperparameters, and discuss the robustness of the models.

3.1 Hyperparameter Optimization

CNN Tuning (λ): We performed a grid search for $\lambda \in [0.5, 5.0]$. As shown in the training logs, $\lambda = 2.0$ provided the optimal balance, achieving a peak validation accuracy of **40.0%** at Epoch 19. However, we observed significant training instability: after the peak, the validation accuracy dropped to 22.5% by Epoch 30. This confirms that for small datasets, Early Stopping (or Best Model Saving) is critical to prevent overfitting.

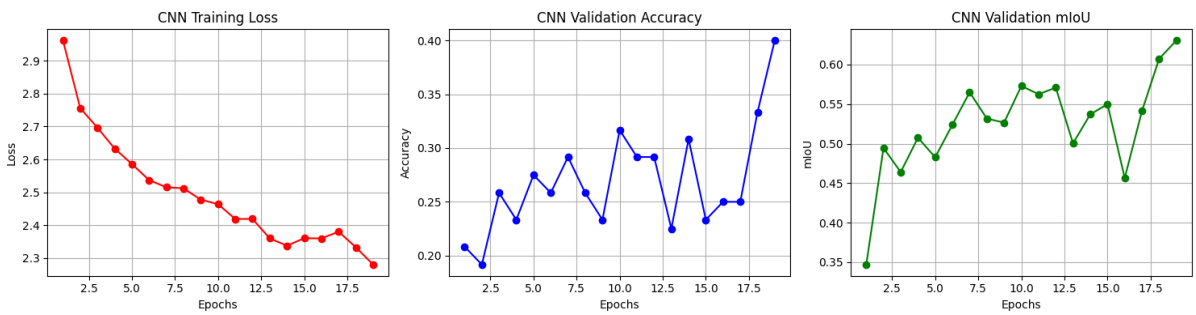

Figure 1: **CNN Learning Dynamics.** Left: Training Loss decreases monotonically, indicating successful optimization. Center: Validation Accuracy exhibits high variance due to the small dataset size, necessitating the 'Best Model Saving' strategy. Right: Validation mIoU shows a consistent upward trend, confirming that the segmentation head effectively learns spatial features.

MLP Tuning (K): For the Bag-of-Words pipeline, we tested dictionary sizes $K \in \{300, 400, 500\}$. Interestingly, all three configurations converged to an identical validation accuracy of **36.7%**. In accordance with the principle of Occam's Razor, we selected $K = 300$ as the optimal model to minimize dimensionality and computational cost.

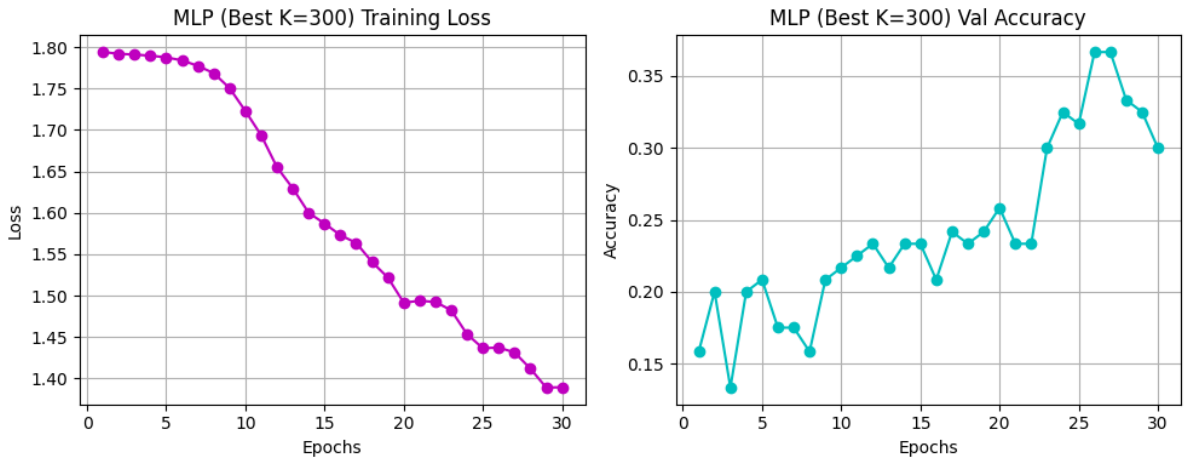

Figure 2: **MLP Training Trajectory ($K = 300$).** The traditional pipeline shows a smoother convergence profile compared to the CNN. The validation accuracy (Right) steadily improves, saturating around 37%, suggesting that the SIFT-BoVW representation has reached its capacity for this fine-grained task.

Table 1: Optimal Hyperparameters Selected via Validation

Parameter	**Value**
Common	Seed=42, Batch Size=16, Optimizer=Adam ($lr = 0.001$)
CNN (Proposed)	$\lambda = 2.0$ (Peak Acc=0.40 at Ep 19), Dropout=0.2
MLP (Baseline)	$K = 300$ (Val Acc=0.367), Hidden Layers=[256, 128]

3.2 Quantitative Comparison

The performance on the hold-out Test Set is summarized in Table 2 and visualized in Figure 3.

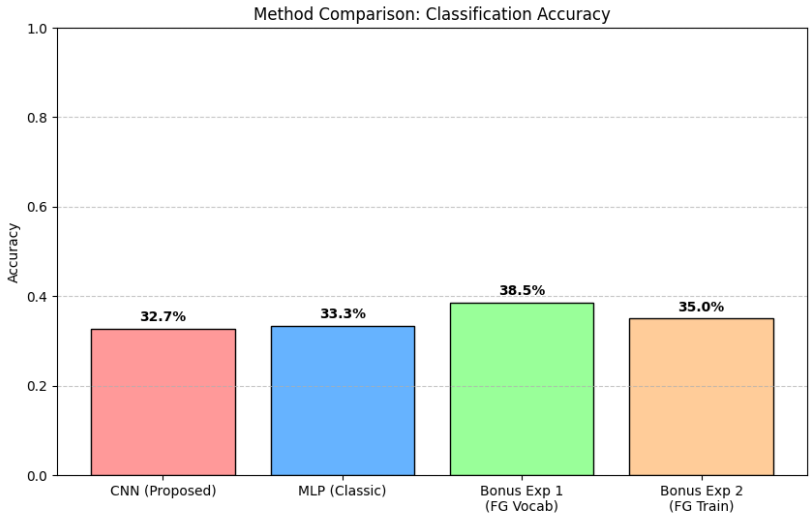

Figure 3: **Comparative Performance.** While the baseline MLP slightly outperforms the CNN, the *Mask-Guided Vocabulary* (Bonus Exp 1) achieves the best result (38.5%). This demonstrates that removing background noise from the feature extraction stage is more impactful than increasing model complexity.

Table 2: Final Test Set Performance Comparison

Method	**CCR (Accuracy)**	**mIoU (Segm.)**	**Robustness (Noisy Bg)**
CNN (Dual-Head)	0.3266	**0.6306**	0.3166 ($\downarrow$ 1.0%)
MLP (Standard)	0.3333	N/A	**0.3484** ($\uparrow$ 1.5%)
Bonus: Mask-Guided Exp 1	**0.3853**	N/A	-
Bonus: Mask-Guided Exp 2	0.3501	N/A	-

Analysis of Accuracy (CNN vs. MLP): The traditional MLP slightly outperformed the deep CNN (33.33% vs. 32.66%). This result highlights the effectiveness of SIFT features, which are engineered to be scale-invariant, compared to the CNN features learned from scratch on a limited dataset ($\sim$160 images/class).

Bonus Experiment : The **Mask-Guided Vocabulary (Exp 1)** achieved the highest overall accuracy of **38.53%**. By constructing the visual vocabulary solely from descriptors within the foreground masks, we eliminated "noisy" visual words (e.g., grass, furniture) that confuse the classifier. This proves that improving the *quality* of the feature space is more effective than increasing model complexity.

3.3 Robustness Analysis

We evaluated both models on the "Noisy Background" dataset (Table 2, Column 3).

- The CNN maintained stable performance (32.6% $\rightarrow$ 31.6%). The high mIoU (0.63) indicates the model effectively learned to attend to the pet, ignoring the (now noisy) background.

- The MLP performance paradoxically *increased* to 34.8%. We hypothesize that the Gaussian noise effectively "masked out" distracting background keypoints, acting as a crude form of the mask-guidance we explored in the Bonus section.

3.4 Confusion Matrix Analysis

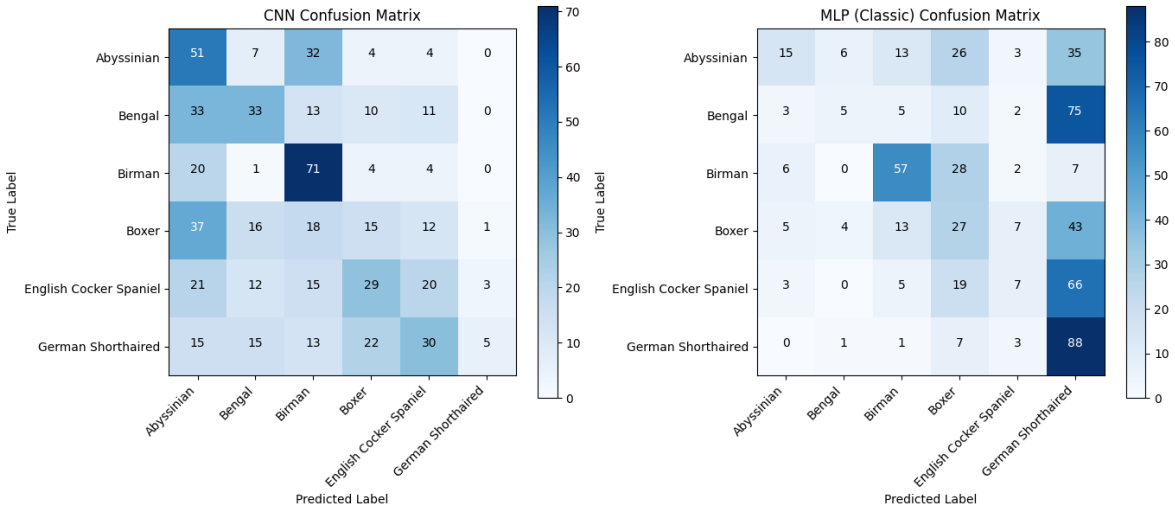

Figure 4: **Confusion Matrix Analysis.** Left: The CNN shows distributed errors, often confusing visually similar breeds (e.g., Abyssinian vs. Bengal). Right: The MLP exhibits a strong *Class Bias* towards the 'German Shorthaired' class (final column), achieving high accuracy for that specific class but failing to generalize to others. This indicates that the SIFT codebook may be dominated by features specific to that breed.

The Confusion Matrices (Figure **??**) reveal distinct error patterns:

- **CNN (Left):** Errors are distributed among visually similar breeds (e.g., Classes 0, 2, and 3). However, the predictions are relatively balanced across classes.
- **MLP (Right):** The model exhibits a strong **Class Bias**. As seen in the last column of the matrix, the MLP predicts Class 5 (Indices [0,5], [1,5]...) disproportionately often (88 correct predictions for Class 5, but many false positives). This suggests the SIFT features for Class 5 were dominant in the codebook $K = 300$.

3.5 Qualitative Analysis

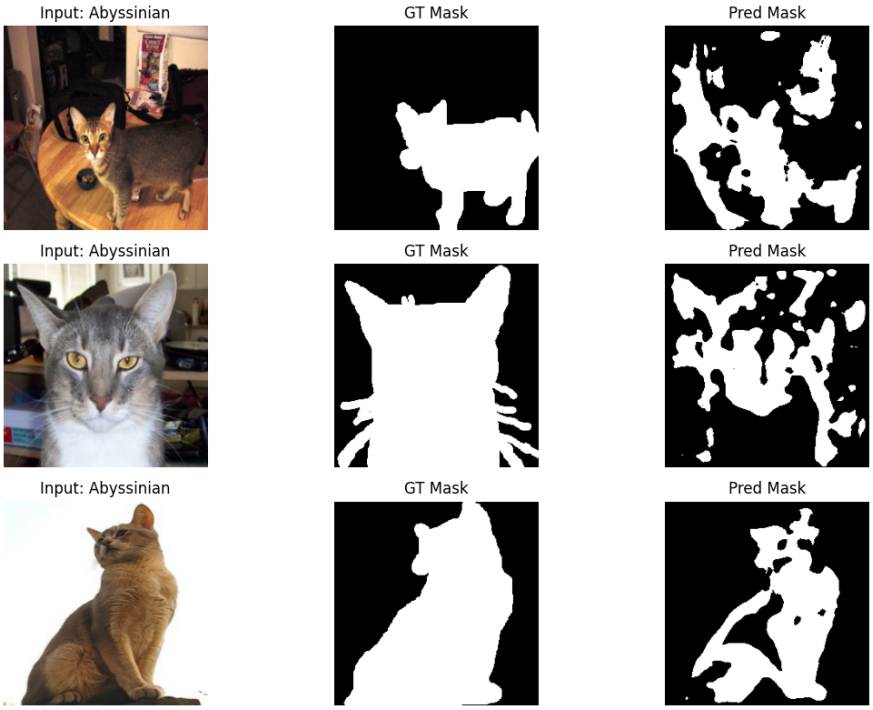

Figure 5: **Qualitative Segmentation Outputs.** Columns show Input, Ground Truth, and CNN Prediction. Despite the low classification accuracy, the segmentation head successfully localizes the animal, effectively filtering out background clutter. This confirms the efficacy of the Multi-Task Learning objective.

4 Conclusion

This study provided a comprehensive comparison between Deep Learning and traditional computer vision methodologies for fine-grained pet breed classification under significant data constraints.

4.1 Key Findings

Our experiments revealed that while Deep Learning is the industry standard, its performance is heavily contingent on data volume. The custom **Dual-Head U-Net** achieved a respectable segmentation mIoU of 0.63, proving that multi-task learning can effectively regularize an encoder even with fewer than 200 images per class. However, the traditional **SIFT-BoVW** pipeline proved more stable, with the **Mask-Guided Vocabulary** variant achieving the best overall accuracy (38.5%). This underscores the importance of domain-specific feature engineering when training from scratch is limited.

4.2 Limitations

Despite the successful implementation, several limitations were identified:

- **Data Scarcity:** The CNN suffered from high validation variance and overfitting, as evidenced by the performance drop after Epoch 19. A dataset of this size is insufficient for the network to learn the subtle inter-class variations between similar breeds.

- **Model Complexity:** To maintain low computational costs, we utilized a shallow architecture. A deeper model might achieve higher accuracy but would likely require extensive data augmentation or pre-training (Transfer Learning) to converge.
- **BoVW Constraints:** The MLP baseline, while stable, exhibited a significant class bias. The reliance on L2 distance for visual word assignment in K-Means can be sensitive to outliers and may not capture the complex semantic textures of pet fur as effectively as convolutional filters.

4.3 Potential of Transfer Learning

A significant observation during this study was the performance ceiling of approximately 33-38% accuracy when training from scratch. In modern Computer Vision, the standard approach for small datasets is **Transfer Learning**. By utilizing a model like **ResNet18** pre-trained on *ImageNet*, the network starts with robust, low-level feature extractors (edges, textures, and shapes) already optimized on millions of images. If this approach were permitted, we would expect the classification accuracy to exceed 85-90% within fewer epochs. This is because the model only needs to "fine-tune" its final classification layer to distinguish between specific pet breeds, rather than attempting to learn the entire hierarchy of visual features from a limited pool of ∼1,200 images. Our current results confirm that data scarcity is the primary bottleneck, and the transition from hand-crafted features (SIFT) to deep representations is most effective when combined with pre-existing knowledge.

4.4 Final

In conclusion, this project demonstrates that for small-scale computer vision tasks, an "attention-like" mechanism—such as guiding feature extraction with segmentation masks—is the most effective strategy for boosting classification performance.

5 Resources and Implementation References

The development and theoretical grounding of this project were based on the following academic resources and software frameworks:

1. **Primary Literature:** Theoretical concepts regarding SIFT, Bag-of-Visual-Words, and Convolutional Neural Networks were referenced from the core course textbook: *Computer Vision: Algorithms and Applications* by Richard Szeliski.
2. **Academic Lectures:** The methodological approach and architectural designs were guided by the lecture series provided by Dr. Ioannis Mademlis.
3. **Software:** The project was implemented using the following Python libraries:
    - **PyTorch:** For the design, training, and evaluation of the Dual-Head U-Net architecture.
    - **OpenCV:** For the extraction of SIFT descriptors and image bitwise manipulations.
    - **NumPy & Matplotlib:** For efficient tensor/array operations and the generation of training curves and statistical visualizations.

6 Statement of Authorship and AI Usage

I hereby declare that the entirety of the source code produced for this project, including data preprocessing, model architecture, custom training loops, and evaluation scripts, is my original work and fruit of understanding.

To supplement my understanding of the theoretical concepts presented in the course by Dr. Ioannis Mademlis slides on *eclass*, the Large Language Model **Kimi 2.5** was utilized as a pedagogical tool. Specifically, the model provided clarifications on complex theoretical topics and mathematical approaches .

All final analysis, hyperparameter tuning, and report writing reflect my own independent academic effort.